

Algorithm2: *Problem Solving* with *Algorithms* and *Data Structures*

Lecture 0.

강의 소개 및 실습 안내

경북대학교 배준현 교수
(joonion@knu.ac.kr)



교과목 개요

개설 학기: 2026년 2학기

교과목명: 알고리즘2 (COMP0320-001)

개설대학: IT대학 컴퓨터학부

학점구조: 3-2-2 (공학전공 – 실험실습)

강의시간/장소: 목요일 1교시-4교시(09:00 – 13:00) / IT대학 5호관 224호

담당교수: 배준현 교수 (joonion@knu.ac.kr)



무엇을 배우는 과목인가?

권장 선수과목: 자료구조/알고리즘¹

권장 후수과목: 고급문제해결

본 과목은 **자료구조/알고리즘¹** 과목에서 배운 개념과 이론을 토대로
주어진 문제를 종합적으로 이해하고 해결할 수 있는 **문제해결 역량을 키우는 것을 목표로 한다.**

따라서, 본 과목은 적절한 자료구조와 알고리즘을 이용한 **문제해결 위주**로 진행한다.

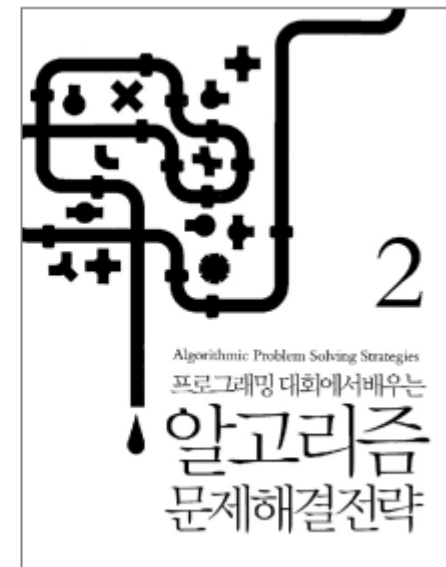


이 과목에서 사용할 교재

(프로그래밍 대회에서 배우는)

알고리즘 문제해결 전략 1. 2.

- 구종만 지음, 인사이트, 2012년.



※ 교재에서는 C++를 프로그래밍 언어로 사용하고 있으나,
본 강의에서는 Python을 프로그래밍 언어로 사용할 것이다.



주별 강의 계획

1주차. 강의 개요 및 실습 안내

2주차. 알고리즘과 문제해결 (Chap. 4+5)

3주차. 브루트포스 (Chap. 6)

4주차. 분할 정복 (Chap. 7)

5주차. 동적 계획법 (1) (Chap. 8)

6주차. 동적 계획법 (2) (Chap. 9)

7주차. 탐욕법 (Chap. 10+31)

8주차. 중간고사

9주차. 트리 & 힙 (Chap. 21+22+23)

10주차. 트라이 & 상호배타적 집합 (Chap. 25+26+31)

11주차. 구간 트리 (Chap. 24)

12주차. 그래프 탐색 (Chap. 28+29)

13주차. 최단 경로 (Chap. 30)

14주차. 조합 탐색(backtracking) (Chap. 11)

15주차. 기말고사

※ 세부적인 강의/실습 일정은 조정될 수 있음.



Lec. 0. 강의 소개 및 실습 안내

평가 방법

출석: 10% (결석 2점, 지각 1점 감점)

결석 4회 이상 F학점 (4회 이상 결석시에도 면담을 통해 합리적 이유가 있으면 구제 가능)

과제: 10%

중간고사: 40%

기말고사: 40%



어떻게 공부할 것인가?

1. 교재: 소장가치가 있는 책이므로 가급적 구매를 권함.
2. 예습: 교재에서 해당 파트를 미리 읽어보고 알고스팟에서 미리 풀어볼 것을 권함.
3. 수업 & 실습
해당 문제를 어떻게 코드로 구현할 것인가를 수업을 통해 이해.
실습 시간에 온라인 저지에서 Python 코드로 문제를 해결.
4. 복습: 유사한 문제를 직접 풀어볼 것.
추천 플랫폼 <https://open.kattis.com/>
5. 생성형 AI 도구 활용
본 수업에서는 생성형 AI의 활용을 적극적으로 권장합니다.
단, 과제를 대신 하게 하는 도구가 아니라 학습을 위한 도구로 활용할 것을 권장합니다.

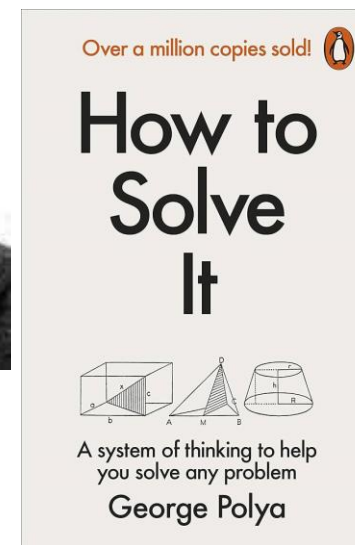


컴퓨터 과학의 본질은 주어진 문제를 컴퓨터를 이용하여 해결하는 것이다.

(문제를) 어떻게 풀 것인가?

How to Solve it? – by George Polya

1. 문제를 이해한다.
2. 어떻게 풀 지 계획을 세운다.
3. 계획을 수행해서 문제를 해결한다.
4. 어떻게 풀었는지 돌아보고,
개선할 방법이 있는지 찾아본다.





컴퓨터 과학자는 어떻게 문제를 해결하는가?

프로그래밍 언어를 사용하여 컴퓨터 프로그램을 작성한다.

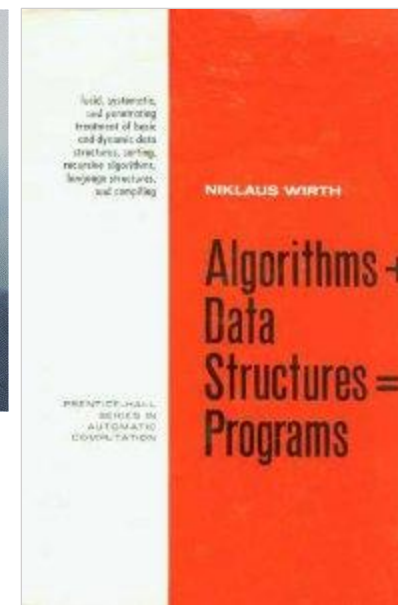
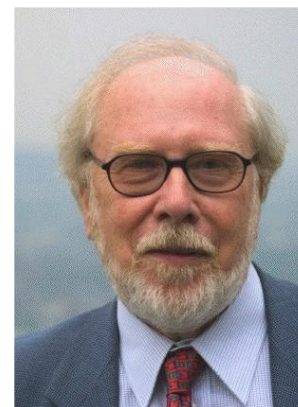
Algorithms + Data Structures = Programs – Niklaus Wirth

알고리즘이란,

주어진 문제를 해결하는 명확한 절차를 말한다.

자료구조란,

데이터를 효율적으로 저장하고 관리하기 위한 체계적 방법을 말한다.





우리가 배운 자료구조들: *Data Structures* Learned

배열과 문자열: *Arrays and Strings*

연결 리스트: *Linked Lists*

스택과 큐: *Stacks, Queues, Deques, Heaps (Priority Queues), etc.*

트리: *Trees* (Binary Search Tree, AVL Tree, Red-Black Tree, etc.)

그래프: *Graphs* (Directed/Undirected, Weighted/Unweighted, etc.)

해시 테이블: *Hash Tables*

기타: *Disjoint Sets, Trie, etc.*



우리가 배운 알고리즘들: *Algorithms* Learned

분할 정복: Binary Search, Merge Sort, Quick Sort, etc.

동적 계획법: Fibonacci Sequence, Binomial Coefficient, Floyd Algorithm, etc.

탐욕적 접근법: Minimum Spanning Tree, Dijkstra, Huffman Code, etc.

완전 탐색: Depth-First-Search, Breadth-First-Search

백트래킹: 0-1 Knapsack, Traveling Salesperson Problem, etc.

기타: *Sorting*, *Searching*, *Hashing*, etc.



(컴퓨터) 알고리즘의 속성

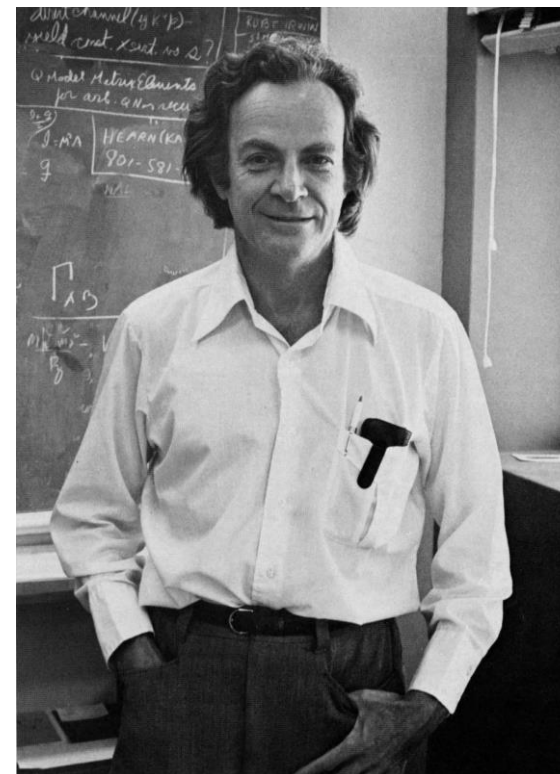
1. 입력(*inputs*): 입력이 없거나, 하나 이상의 입력을 필요로 한다.
2. 출력(*outputs*): 적어도 하나 이상의 출력을 생성한다.
3. 유한성: *finiteness*
알고리즘은 유한한 단계 내에서 반드시 종료해야만 한다.
4. 명확성: *unambiguity, definiteness*
각 단계는 무엇을 해야 할 지 분명하게 알 수 있을 정도로 명확해야 한다.
5. 효과성(실행가능성): *feasibility, effectiveness*
각 단계는 (컴퓨터가) 실행할 수 있는 절차로 구성되어야 한다.



파인만 알고리즘은 궁극의 알고리즘인가?

FEYNMAN-ALGORITHM:

1. Write down the problem.
2. *Think real hard.*
3. Write down the solution.





컴퓨팅 사고력과 문제해결 능력을 키우는 가장 좋은 방법은?

좋은 문제를, 스스로의 힘으로, 많이 풀어보는 것이다.

1. **좋은 문제**: 주로 교과서, 교재 등에 등장하는 문제들

2. **스스로의 힘으로** 풀어보기:

스스로의 힘으로 **풀 수 있는 힘을 키운 뒤**(교재, 수업 등을 통한 학습),

반드시 스스로의 힘만으로 **새로운 문제**를 풀어보아야 한다.

3. **많이 풀어보기**: 온라인 저지 서비스 활용

BOJ(백준 온라인 저지): 가장 많은, 최신의 문제를 접할 수 있음

LeetCode: 알고리즘/자료구조의 전형적인 문제를 접할 수 있음(코딩 테스트/인터뷰 대비용)

AlgoSpot: 교재의 문제를 온라인에서 풀어볼 수 있음 (Python 무렵)



Lec. 0. 강의 소개 및 실습 안내

실습용 온라인 저지 사용법

주소: <http://155.230.35.171>

아이디/패스워드 생성 요령:

아이디는 본인 **한글 실명**으로 생성하되, 동명이인이 있을 경우 이름 뒤에 학번이나 문자 추가

예) 홍길동²², 홍길동²³, 홍길동^{24a}, 홍길동^{24b}

패스워드는 노출되어도 무관한 것으로 사용: 개인 서버이므로 보안 관리 안됨

아이디/패스워드 분실시:

위에서 설명한 방법으로 새로운 아이디를 만들 것 (이메일 전송 안됨)



Lec. 0. 강의 소개 및 실습 안내

실습 과제 수행법

매주 이론 강의를 마친 후 Online Judge > *Contests* 에 해당 주차의 과제가 생성됨.

예) 알고리즘2: 실습과제 00

입장 비밀번호: ALGO2

실습 과제는 특별한 사정이 없는 한, 수업 시간 이내에 모두 풀어야 함.





Lec. 0. 강의 소개 및 실습 안내

온라인 저지의 파이썬 버전 확인

알고스팟: *Python 3* (3.4.3)

f-string(3.6부터 가능) 등은 사용 불가

실습용 온라인 저지: *Python 3* (3.12)

f-string(3.6+) 사용 가능, 딕셔너리의 key 순서 보장(3.7+)



Lec. 0. 강의 소개 및 실습 안내

(권장) 실습 환경 구축

통합개발환경: Visual Studio Code + Python 최신 버전 (3.12.x)

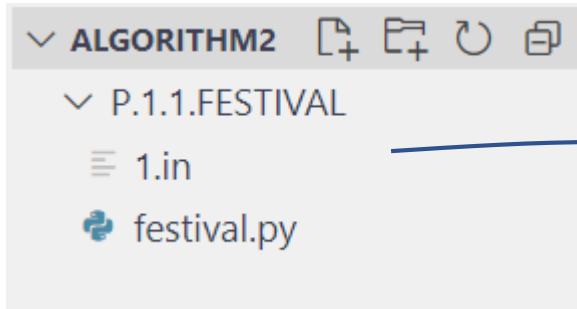


파이썬 설치시 환경변수 설정을 할 것.

Add python.exe to PATH



Lec. 0. 강의 소개 및 실습 안내



문제 풀이를 위한 폴더 생성 후 입력 파일과 코드 파일 생성

입력 파일: 1.in

코드 파일: festival.1.py

폴이가 여러 개이므로,
일련 번호를 붙여 두면 좋다.

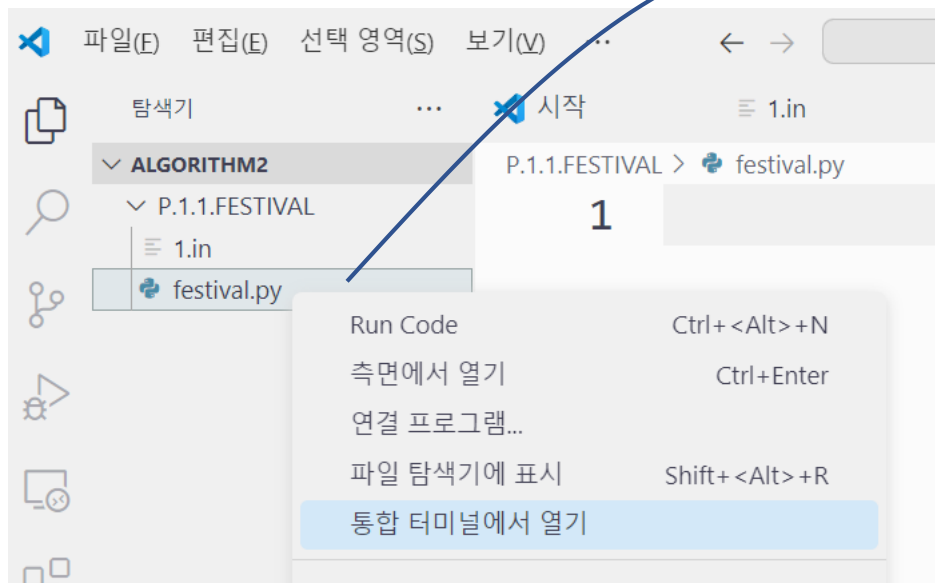


Lec. 0. 강의 소개 및 실습 안내

Sample Input 1 

```
4 16
2 5 10 5
40 30 50 10
```

온라인 저지에서 예제 입력을 copy하여
1.in에 붙여넣기하고 저장



소스 파일에서 오른쪽 마우스 클릭 후
“통합 터미널에서 열기”를 선택하여
명령어 프롬프트 실행



Lec. 0. 강의 소개 및 실습 안내

```

> 터미널
D:\code\knu-algorithm2\Lab.01.Overview\P.1.1.FESTIVAL>py festival.1.py < 1.in
1.75
1.5
D:\code\knu-algorithm2\Lab.01.Overview\P.1.1.FESTIVAL>
  
```

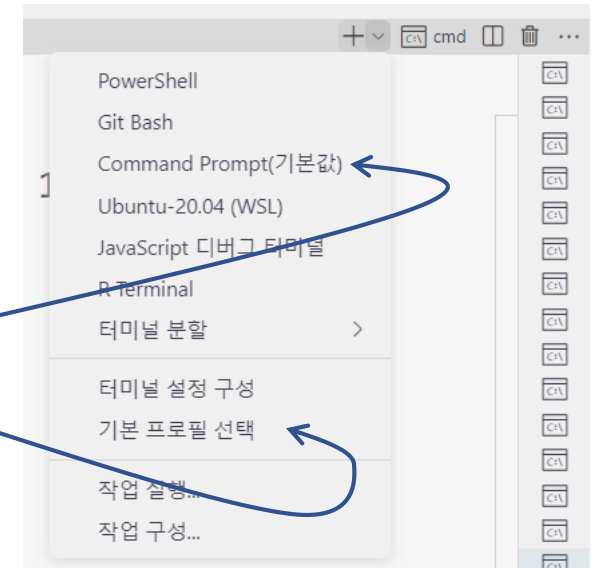
코드를 작성후 **명령어 프롬프트**에서 실행

Power Shell에서는 리다이렉션 안 됨

→ 실행 결과 확인

Command Prompt를

기본 터미널로 설정해 두면 편리함





Lec. 0. 강의 소개 및 실습 안내

AL2.0.1: 동전 뒤집기 1 (ID: COINFLIP1)

문제

N개의 동전이 일렬로 놓여 있다.

각 동전은 앞면(H) 또는 뒷면(T)이 위를 향하고 있다.

한 번의 뒤집기로 서로 인접한 두 동전을 선택하여 두 동전을 동시에 뒤집을 수 있다.

뒤집기를 원하는 만큼 수행할 수 있을 때,

주어진 상태에서 모든 동전을 앞면(H)으로 만들 수 있는지 판단하시오.



Lec. 0. 강의 소개 및 실습 안내

입력

첫째 줄에 동전의 개수 N 이 주어진다.

둘째 줄에 N 개의 동전 상태를 나타내는 길이 N 의 문자열 S 가 주어진다.

$$1 \leq N \leq 1,000,000$$

출력

모든 동전을 앞면으로 만들 수 있으면 "YES"를 출력한다.

모든 동전을 앞면으로 만드는 것이 불가능하면 "NO"를 출력한다.



Lec. 0. 강의 소개 및 실습 안내

예제 입력

4
THHT

5
HTHTH

4
HTHH

6
HHHHHH

예제 출력

YES

YES

NO

YES



Lec. 0. 강의 소개 및 실습 안내

```
import sys
input = sys.stdin.readline

def solve(coins):
    if coins.count("T") % 2 == 0:
        return "YES"
    else:
        return "NO"

N = int(input())
coins = input().strip()
print(solve(coins))
```



AL2.0.2: 동전 뒤집기 2 (ID: COINFLIP2)

문제

N개의 동전이 일렬로 놓여 있다.

각 동전은 앞면(H) 또는 뒷면(T)이 위를 향하고 있다.

한 번의 뒤집기로 서로 인접한 두 동전을 선택하여 두 동전을 동시에 뒤집을 수 있다.

주어진 상태에서 모든 동전을 앞면(H)으로 만들 수 있다면, 필요한 최소 뒤집기 횟수를 구하시오.

모든 동전을 앞면으로 만드는 것이 불가능하다면 -1을 출력한다.



Lec. 0. 강의 소개 및 실습 안내

예를 들어, THHT 가 주어졌다고 하자.

다음과 같이 동전을 뒤집을 수 있다.

1. THHT

2. HTHT

3. HHTT

4. HHHH

따라서 모든 동전을 앞면으로 만들기 위한 최소 뒤집기 횟수는 3이다.



Lec. 0. 강의 소개 및 실습 안내

입력

첫째 줄에 동전의 개수 N 이 주어진다.

둘째 줄에 N 개의 동전 상태를 나타내는 길이 N 의 문자열 S 가 주어진다.

$$1 \leq N \leq 1,000,000$$

출력

모든 동전을 앞면으로 만들 수 있다면 필요한 최소 뒤집기 횟수를 출력한다.

모든 동전을 앞면으로 만드는 것이 불가능하다면 -1 을 출력한다.



Lec. 0. 강의 소개 및 실습 안내

예제 입력

4
THHT

4
TTHH

5
HTHTH

4
HTHH

6
HHHHHH

예제 출력

3

1

2

-1

0



```
def solve(coins):  
    unmatched_tail = -1  
    total = 0  
  
    for index, state in enumerate(coins):  
        if state == "T":  
            if unmatched_tail < 0:  
                unmatched_tail = index  
            else:  
                total += index - unmatched_tail  
                unmatched_tail = -1  
  
    return -1 if unmatched_tail >= 0 else total
```

Any Questions?

